MacZine

THE MACTECH SOLUTIONS NEWSLETTER
◆ · JULY 2026 · MACTECHSOLUTIONSLLC.COM/MACZINE

HOW WE PUBLISH

Why MacZine Publishes From a Git Repository

Every MacZine article is a Markdown file, reviewed as a pull request, and live within seconds of merge. Here's why we built the newsletter that way.

BY **MACTECH SOLUTIONS** READING TIME · 4 MIN MACZINE · PUBLISHING · TRANSPARENCY

This article is a Markdown file in a public GitHub repository, and the pull request that added it is the only editorial review it got. That is not an incidental detail of how MacZine is built — it is the point.

FIELD COPY
JULY 2026
MACZINE

4 min

READING TIME, NO PAYWALL, NO POP-UPS

◆ ONE SERIES, ONE LINTER

Every published article carries a unique `issue:` number, assigned in publication order. A script checks every pull request and fails the build on a missing number, a duplicate one, a missing field, or an oversize title — the catalog rule is enforced by code, not remembered by an editor.

The repo is the newsletter

MacZine has no content management system. The repository `MacTech-Solutions-LLC/maczine` *is* the source of truth: every article lives at `articles/<slug>/index.md`, plain GitHub-flavored Markdown with a YAML frontmatter block for title, description, publish date, tags, and a few optional fields that feed the site's margin rail. There is no separate draft database and no admin panel where a piece can exist without a commit behind it. If it's not in the repo, it isn't published, and if it's in the repo, its whole history — every revision, every author, every timestamp — is sitting in `git log` for anyone to read.

Drafts are branches, review is a pull request

Writing an article means branching, adding the file, and opening a pull request against `main`. That PR is the entire editorial process: it's where a reviewer reads the draft, where CI runs a linter against the frontmatter, and where the diff — this exact set of added lines, nothing more, nothing less — gets a green check or a requested change. There is no separate "approved" state that lives outside version control. The review *is* a piece of the commit graph, permanently attached to the words it approved.

The catalog, enforced by code

MacZine runs one numbered series: every article that publishes carries an `issue:` field, assigned in publication order, so Issue Nº 001 is the first piece the newsletter ever ran and the newest issue always holds the highest number. A number is earned at publish, not at authorship — a draft carries none until it goes live. That isn't a style-guide convention an editor has to remember to apply. `scripts/lint-articles.mjs` runs on every pull request and fails the build if an article publishes without a number, if a number repeats, if a required frontmatter field is missing, if `tags` isn't a list, or if the body smuggles in a stray `#` heading that would collide with the page's own h1. The catalog stays unambiguous because a script — not a style guide — refuses to let it drift.

Merge to main, live in seconds

Once a PR is approved and merged, a GitHub webhook fires against the site's sync endpoint, which mirrors `articles/` into the live database and revalidates the affected pages. There is no separate deploy step and no release train to wait on — the same merge that closes the pull request is the publish action. A second, independent workflow then renders a press-room field copy: a print-style PDF of the article, built by `scripts/build-article.mjs` from the same Markdown and frontmatter, and committed back into the article's own directory. The rendered artifact — web page and PDF alike — always traces back to one reviewed commit.

Document-as-you-build, applied to ourselves

The habit that separates programs that pass an assessment from programs that panic is the same one that runs this newsletter: evidence gets captured when the work happens, not reconstructed afterward.

We tell clients building CMMC Level 2 programs to update their System Security Plan in the same change window as the system it describes, because a document written after the fact is a reconstruction, not a record. MacZine runs on the identical discipline, pointed at ourselves: the article, the review, and the publish action are one artifact instead of three. We're not asking readers to trust a process we don't use.

What you can audit

Because the repository is public, none of this is a claim you have to take on faith. The commit history shows who wrote which sentence and when. The pull requests show what a reviewer flagged before merge. The lint script and its rules are readable in `scripts/lint-articles.mjs`. The publishing workflows are readable in `.github/workflows/`. If an article changes after publication, that's a new commit, visible in the same history as everything else — there's no quiet edit that leaves no trace.

Why this matters to a defense-industrial-base audience

Readers of this newsletter spend their working hours building evidence trails for assessors: SSPs, POA&Ms, SPRS scores, audit logs that have to hold up to a C3PAO or a contracting officer asking "prove it." A publication that can't show its own work has no standing to lecture anyone about evidence discipline. MacZine's editorial process is a public git history because the alternative — asking you to trust an opaque CMS we don't show you — is exactly the kind of unverifiable claim we tell clients to eliminate from their own compliance posture. If you want to see what a reviewed, versioned, timestamped process looks like before you build one for an assessor, the repo is open. If you'd rather we build that discipline into your program directly, start a readiness conversation or get in touch. ◆